

IA353 – Redes Neurais (1s2026)
Projetos Computacionais – Parte 2 – PCs #06 a #13
Atividade em Dupla – Peso Total: 7,0
Data de entrega dos resultados solicitados: 25/06/2026

Projeção linear e não-linear, Autoencoders, Redes de Hopfield e de Kohonen e Deep Learning

1. Projeto Computacional #06 (peso 1,0): Visualização de dados por projeções lineares e não-lineares

Olhar diretamente para os dados pode ser muito útil e geralmente é considerado um dos primeiros passos na análise exploratória de dados. Mas como visualizar dados caracterizados por uma grande quantidade de atributos, ou seja, dados que são pontos em espaços de alta dimensão? Uma possível resposta está nas projeções 2D e 3D (lineares ou não-lineares) que buscam preservar o máximo possível da topologia original dos dados. A melhor e mais popular técnica de projeção linear para dados não rotulados é a Análise de Componentes Principais (do inglês *Principal Component Analysis* – PCA) e, para dados rotulados, uma boa opção é Análise Discriminante Linear (do inglês *Linear Discriminant Analysis* – LDA). Mas no tocante a projeções não-lineares, há muitas opções, cada qual com seus pontos fortes e fracos. Vamos considerar, nesta atividade, t-SNE (do inglês *t-Distributed Stochastic Neighbor Embedding*), UMAP (do inglês Uniform Manifold Approximation and Projection) e mapas auto-organizáveis (SOM, do inglês Self-Organizing Maps), embora autoencoders e outras técnicas de PCA não-linear também possam ser adotadas. **A estratégia para PCA, t-SNE e UMAP é realizar projeções sem levar em conta o rótulo dos dados, mas, na sequência, apresentar os resultados com a devida rotulação, usando cores para indicar as classes.** A ênfase vai estar na técnica UMAP, por ser bem flexível, mais escalável que as concorrentes (de projeção não-linear) e dotada de uma tendência destacada em preservar grupos. No entanto, UMAP é sensível à definição de hiperparâmetros, como o número k de vizinhos. O vídeo em [<https://www.youtube.com/watch?v=eN0wFzBA4Sc>] apresenta uma explicação didática e sucinta do princípio de operação da técnica, além de a comparar (brevemente) com t-SNE. Assista a este vídeo, execute o notebook [PC06_lin_nonlin_dim_reduct.ipynb] e responda às 2 questões apresentadas em cor verde. São considerados dois casos de estudo (Iris e MNIST). Tempo total de execução do código: em torno de 10 minutos.

2. Projeto Computacional #07 (peso 1,0): Codificador, decodificador e código em autoencoders

Este projeto envolve a captura de um manifold 2D de uma base de dados de dimensão bem mais elevada. Tome a base de dados MNIST, realize o treinamento de um *autoencoder* com camadas densas e constate o sucesso da tarefa de codificação e de decodificação (mesmo que ainda com algumas imperfeições), indicando que duas variáveis latentes podem ser suficientes para codificar a essência de toda a variedade exibida pelas versões manuscritas dos 10 dígitos. Por fim, explore o *manifold* gerado no gargalo do

autoencoder. Para tanto, execute o notebook [PC07_P1_AE_MNIST.ipynb] e procure compreender o que está sendo feito em cada trecho de código (não é preciso reportar esta atividade de compreensão das funcionalidades envolvidas, bastando apresentar o notebook executado). Para as células [Moving along a circle], [Moving along an ellipse] e [Moving along a grid], procure ajustar o intervalo de excursão para explorar melhor a “nuvem de pontos” 2D. Em seguida, execute o notebook [PC07_P2_AE_MNIST.ipynb], que implementa o *autoencoder* utilizando agora camadas convolucionais, com *downsampling* no codificador e *upsampling* no decodificador, realizando as 4 atividades requeridas. Tempo total de execução do código, incluindo as Partes 1 e 2: em torno de 10 minutos.

3. Projeto Computacional #08 (peso 0,5): Rede de Hopfield para o TSP

Mesmo ciente de que, neste momento de evolução tecnológica, existem estratégias mais eficientes para resolver o problema do caixeiro-viajante, como o Concorde TSP Solver [<https://www.math.uwaterloo.ca/tsp/concorde.html>], execute o notebook [PC08_HN_TSP.ipynb] para obter soluções de caixeiro-viajante para as instâncias disponíveis, apresentando o melhor resultado em cada caso. A seguir, responda às 2 questões levantadas. Tempo total de execução do código: em torno de 8 minutos.

4. Projeto Computacional #09 (peso 0,5): Rede de Kohonen para o TSP

Com a mesma ressalva feita no enunciado do PC08, execute o notebook [PC09_SOM_TSP.ipynb] para obter soluções de caixeiro viajante empregando um mapa de Kohonen. Para os dois primeiros casos de estudo, é necessário carregar os arquivos [qa194.tsp] e [pcb1173.tsp], fornecidos pelo professor. Fique à vontade para escolher o terceiro caso de estudo, dentre as opções em:

<https://comopt.ifl.uni-heidelberg.de/software/TSPLIB95/tsp.html>

Nesta atividade, as instâncias acabam sendo bem maiores que aquelas do PC08.

Tempo total de execução do código: em torno de 7 minutos.

5. Projeto Computacional #10 (peso 1,0): Comparação entre modelos de predição de séries temporais

A predição de séries temporais pode ser realizada por meio de um grande número de técnicas, com tratamentos específicos para a série temporal e com objetivos variados. Nesta atividade, vamos empregar redes neurais recorrentes (SimpleRNN, LSTM e GRU) e não-recorrentes (Preditor Linear, MLP e CNN) na predição de duas séries temporais: Sunspot (média mensal de 1749 até 1983) e uma série sintética não-linear (TS). As predições podem ser de um passo à frente ou de múltiplos passos à frente, mas não é praticada aqui a predição recursiva (com a realimentação da predição feita como entrada para a predição seguinte). Para efeito de comparação, entre as abordagens recorrentes e não-recorrentes, o número de desdobramentos no tempo e o número de atrasos (*tapped delay line*) serão os mesmos. As séries temporais sofrem apenas um escalamento, embora outros tratamentos, como eliminação de tendências, pudessem ser considerados. Não cabe estender os resultados das análises comparativas a outras séries temporais, pois cada série tem suas particularidades e vai desafiar os modelos de aprendizado de formas distintas. Os hiperparâmetros dos diversos modelos de predição não passaram por um processo de sintonia fina. Execute o notebook [PC10_TSprediction.ipynb] para os dois casos de estudo: Sunspot e TS.

Apresente, assim, dois notebooks em seu relatório, sendo que **edições são requeridas na segunda e na quarta célula de código** (na hora de decidir que arquivo CSV será lido). Responda às 5 questões das células finais em apenas um dos dois notebooks a serem entregues, mesmo que envolva resultados dos dois casos de estudo.

6. Projeto Computacional #11 (peso 1,0): Tradução de frases usando dois blocos LSTM

Nesta atividade, vamos implementar um Encoder-Decoder LSTM para executar uma tarefa análoga à tradução de frases realizada por vários aplicativos da internet, mas envolvendo bem menos “palavras” das duas linguagens. Trata-se de um preditor de uma sequência de saída a partir de uma sequência de entrada (mapeamento seq2seq). Execute o notebook [PC11_PLN_LSTM.ipynb] e explique como são gerados os dados de treinamento. Em seguida, explique como uma calculadora simples pode operar baseada no conceito de tradução de frases, ou seja, sem realizar operações algébricas. Este notebook está baseado no material presente em [Brownlee, J. “Long Short-Term Memory Networks With Python – Develop Sequence Prediction Models With Deep Learning”, Machine Learning Mastery, 2017], mais especificamente no [Chapter 9].

7. Projeto Computacional #12 (peso 1,0): Generative Adversarial Networks (GANs)

O treinamento de redes neurais adversárias foi proposto em 2014:

I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, Y. Bengio, **Generative adversarial nets**, Advances in Neural Information Processing Systems, pp. 2672-2680, 2014.

A exploração de todo o seu potencial requer muita memória e muito processamento, algo que não será exigido neste projeto. Busca-se executar o treinamento de uma GAN para reproduzir dígitos manuscritos a partir da base MNIST. O objetivo é executar o notebook [PC12_GAN_MNIST.ipynb]. O código executa o treinamento por 20.000 épocas e salva exemplos de imagens a cada 1.000 épocas. Execute o código e exiba: a primeira imagem obtida, além de imagens obtidas após 1.000, 10.000 e 20.000 épocas. Forneça respostas às duas questões levantadas. Como material de apoio, considere:

<https://machinelearningmastery.com/how-to-develop-a-generative-adversarial-network-for-a-cifar-10-small-object-photographs-from-scratch/>

Tempo total de execução do código: em torno de 70 minutos.

8. Projeto Computacional #13 (peso 1,0): Transformer em miniatura operando como um chatbot

Os *transformers* representam uma das arquiteturas mais importantes na área de processamento de linguagem natural, tendo revolucionado a forma como as máquinas aprendem padrões linguísticos, interpretam o contexto e geram texto coerente. Diferentemente das redes neurais recorrentes tradicionais, que processam sequências de maneira estritamente temporal, os *transformers* utilizam mecanismos de atenção (*attention mechanisms*) capazes de analisar simultaneamente todas as palavras de uma

sequência, permitindo maior paralelismo computacional e melhor modelagem de dependências de longo prazo. Essa característica tornou os *transformers* particularmente adequados para aplicações que envolvem tradução automática, sumarização de textos, sistemas de perguntas e respostas e, especialmente, *chatbots* inteligentes.

Quando um *transformer* opera como um *chatbot*, sua principal função é receber uma sequência de palavras fornecida pelo usuário e gerar, palavra por palavra, uma resposta textual plausível e contextualizada. O núcleo do *transformer* é composto por múltiplos blocos empilhados contendo mecanismos de atenção multi-cabeça (*multi-head attention*), redes neurais feedforward, conexões residuais e procedimentos de normalização. Após treinado, o *transformer* passa a operar em modo de inferência, gerando respostas palavra por palavra a partir do histórico textual da conversa. Embora o modelo não possua compreensão consciente da linguagem, ele é capaz de aprender estruturas estatísticas extremamente sofisticadas, produzindo respostas frequentemente coerentes, contextualizadas e semanticamente plausíveis. Dessa forma, os *transformers* consolidaram-se como a principal base tecnológica dos sistemas conversacionais modernos, incluindo assistentes virtuais e grandes modelos de linguagem contemporâneos.

Execute o notebook [**PC13_transformer.ipynb**], observe os resultados obtidos e responda às 4 questões apresentadas ao final.